Program Analysis with ML

ML, Coding Companions, The Future

Yegor Bugayenko

Lecture #10 out of 10
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

- How Machine Learning Works?
- AI Coding Companions
- What's Next?

Chapter #1:

How Machine Learning Works?

Deterministic Algorithm

A deterministic algorithm means that given a particular input, the algorithm will always produce the same output:

```
String sayHello(int hours, int minutes) {
  if (hours > 4 && hours < 12) {
    return "Good morning!";
  } else if (hours > 12 && hours < 18) {
    return "Good afternoon!";
  } else if (hours > 18 && hours < 22) {
    return "Good evening!";
  }
  return "Good night!";
}
```

The behavior of the `sayHello()` function is defined upfront by its creator.

Training a Model

"The process of training an ML model involves providing a learning algorithm with training data. The algorithm finds patterns in the training data that map the input data attributes to the target attributes, and it outputs an ML model that captures these patterns." (c) Amazon

```
 8:35  Good morning!
 8:40  Good morning!
10:00  How are you?
11:55  Good afternoon!
13:18  Good day!
14:50  Good afternoon!
15:22  Good afternoon, Sir!
17:14  Good evening!
...
22:34  Evening!
23:50  Good night!
```

$$f : H \times M \to G$$

$$H = \{0, 1, 2, \ldots, 23\}$$

$$M = \{0, 1, 2, \ldots, 59\}$$

$$G = \{\texttt{"Good morning!"}, \texttt{"Good afternoon!"}, \texttt{"Good evening!"}, \texttt{"Good night!"}\}$$

Embedding

"An embedding is a relatively low-dimensional space into which you can translate high-dimensional vectors." (c) Google

Features:		Embeddings:	Vectors:
8:35	Good morning!	(8, 30, "morning")	(8, 30, 0)
8:40	Good morning!	(8, 45, "morning")	(8, 45, 0)
10:00	How are you?	NIL	
11:55	Good afternoon!	(12, 00, "morning")	(12, 00, 0)
13:18	Good day!	NIL	
14:50	Good afternoon!	(14, 45, "afternoon")	(14, 45, 1)
15:22	Good afternoon, Sir!	(15, 30, "afternoon")	(15, 30, 1)
17:14	Good evening!	(17, 00, "evening")	(17, 00, 2)
...		...	...
22:34	Evening!	(22, 30, "evening")	(22, 30, 2)
23:50	Good night!	(23, 45, "night")	(23, 45, 3)

Data Science

There are five tasks to complete repeatedly:

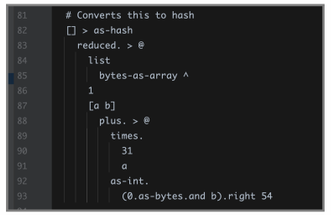
1. Collect and clean a dataset
2. Define features + embeddings
3. Choose and tune an algorithm
4. Train a Model
5. Validate the Model

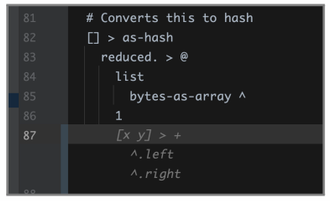
Chapter #2:

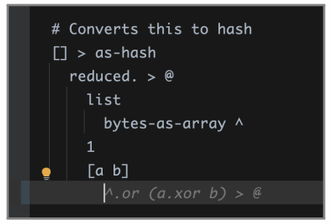
AI Coding Companions

Auto Code Completion

This is how Copilot by GitHub is suggesting code completion in our own programming language, which he definitely hasn't seen before:



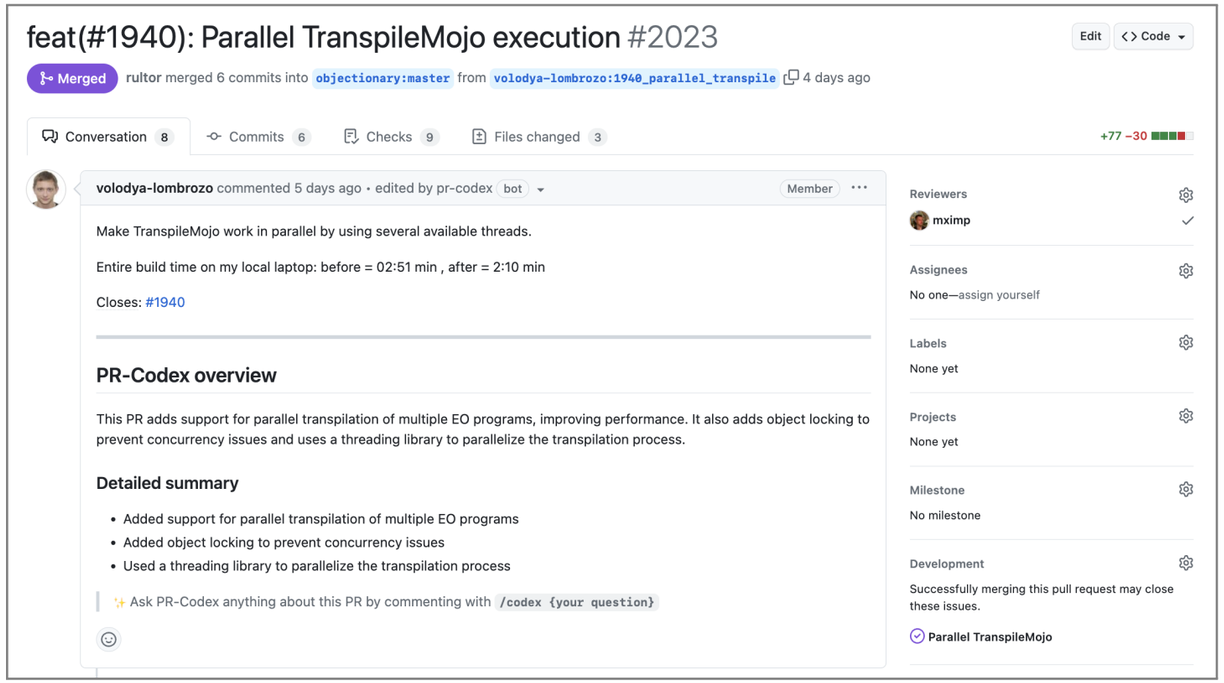






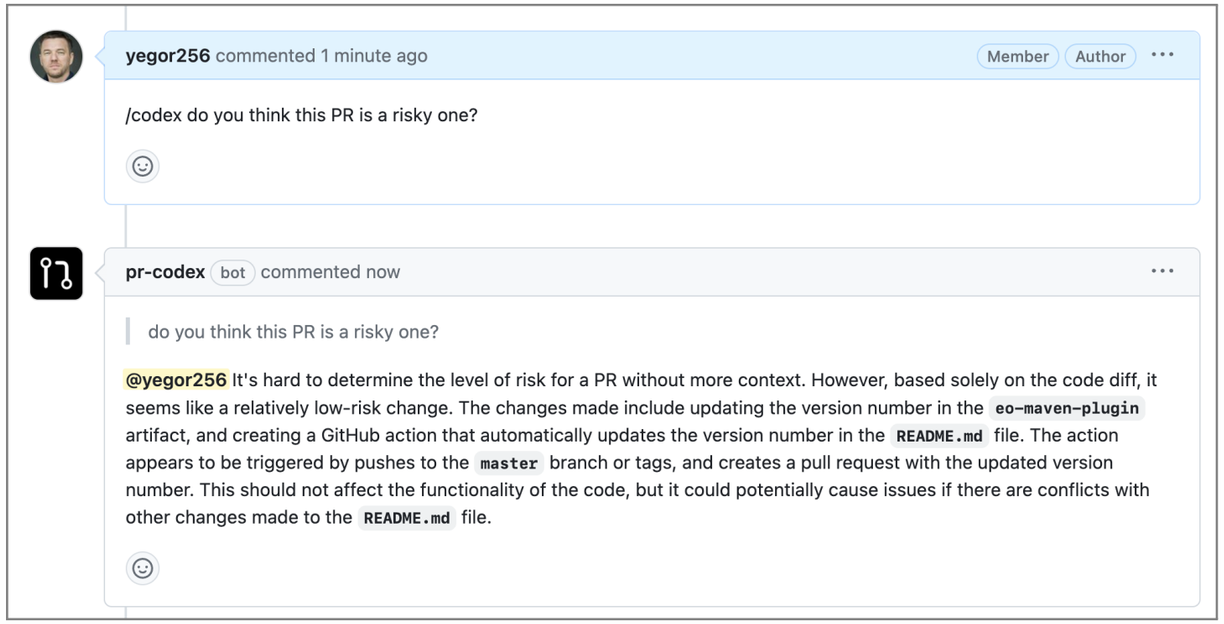
Read also about AWS CodeWhisperer at Werner Vogels' blog. Also, about TabNine (used to be Codota).

Pull Request Explanation



PR-Codex plugin for GitHub by dlabs

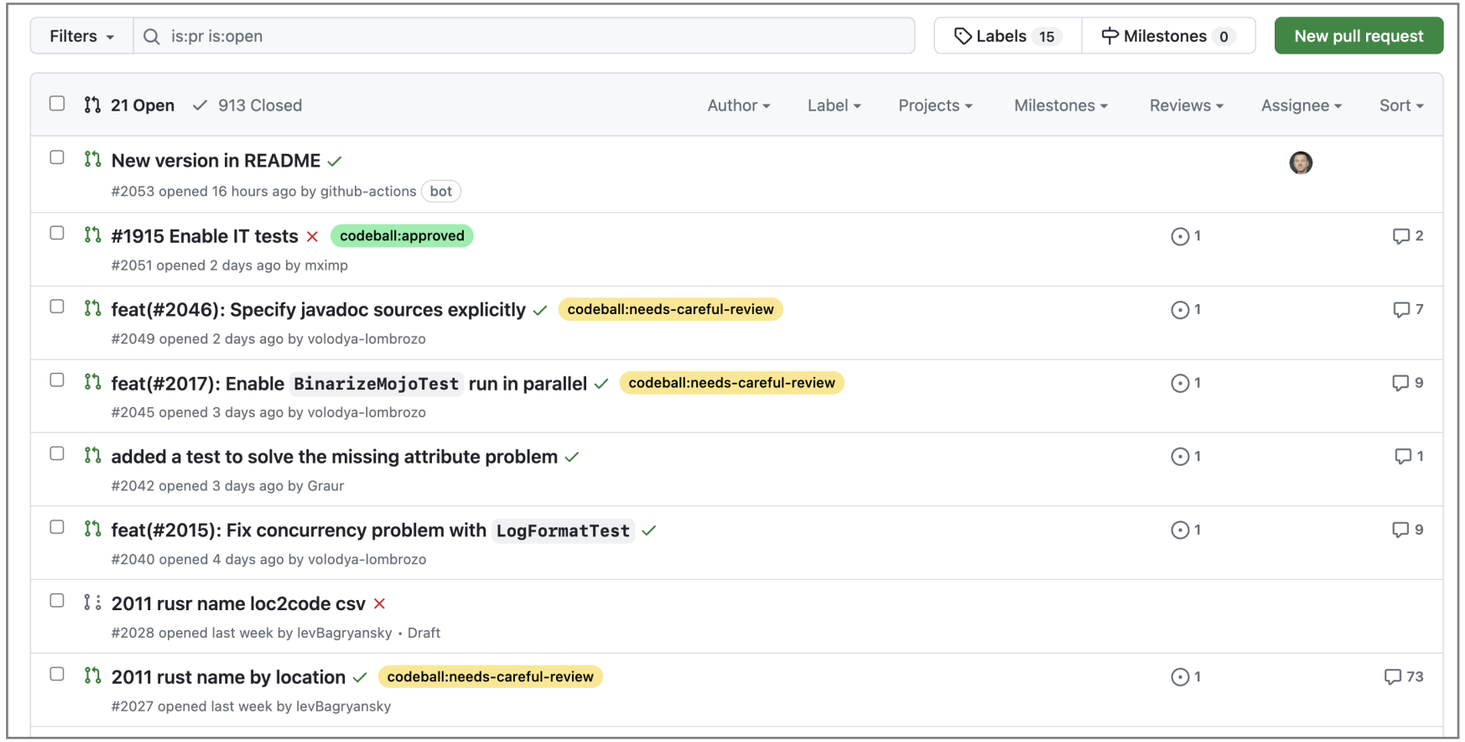
Reviewing Changes



The discussion happened in this GitHub issue: objectionary/eo:2034

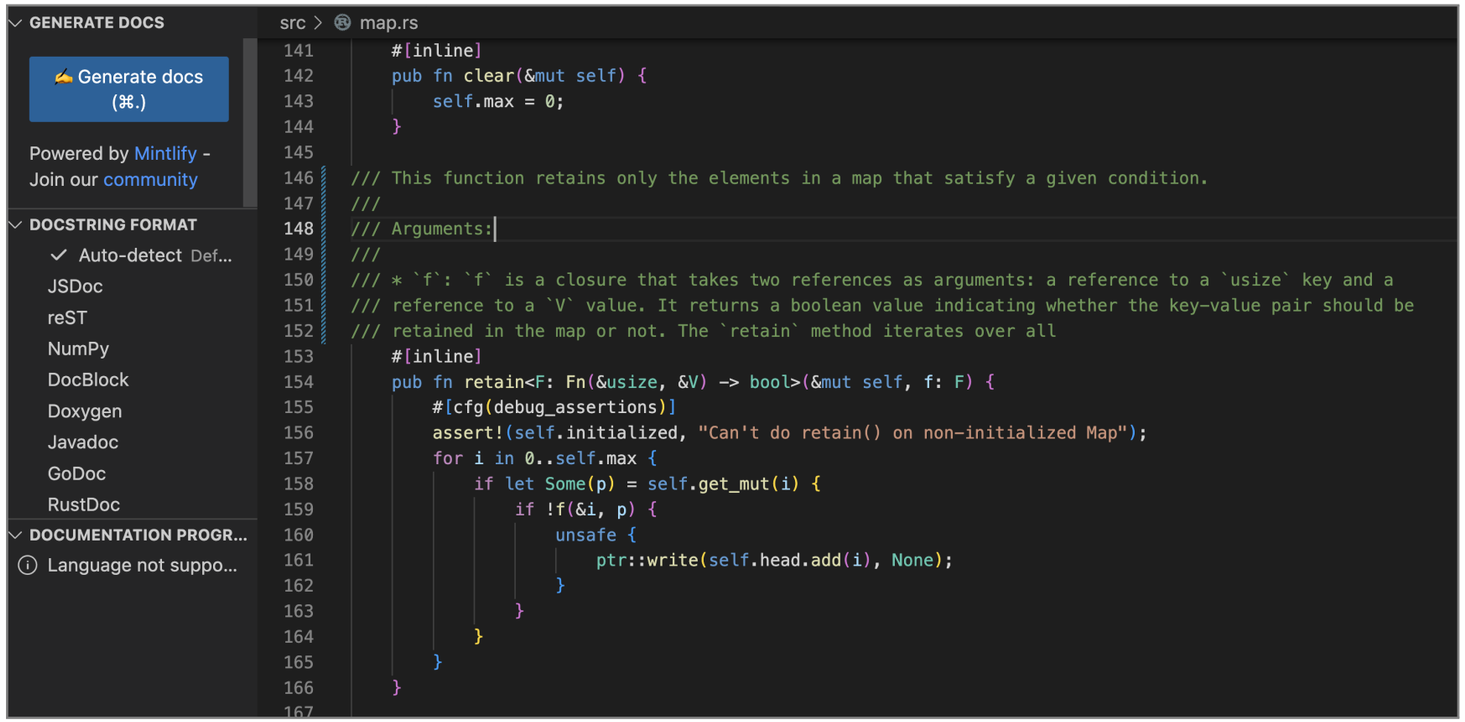
[Complete PR Review Risks Explain Safurai Repeat Tests Refactor LLM]

Pull Request Risk Analysis



Codeball plugin for GitHub

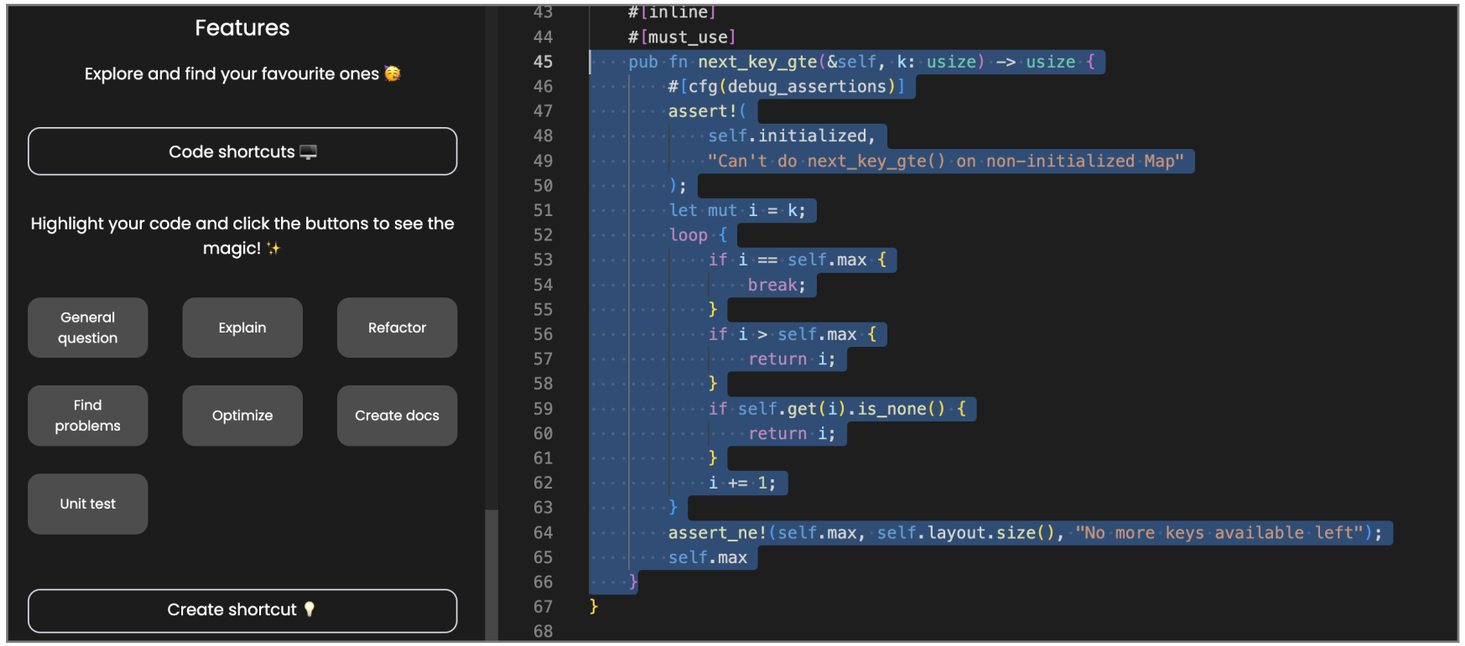
Explain This to Me!



Writer plugin for VS-Code by Mintlify

[Complete PR Review Risks Explain Safurai Repeat Tests Refactor LLM]

All in One Package



Plugin for VS-Code by Safurai

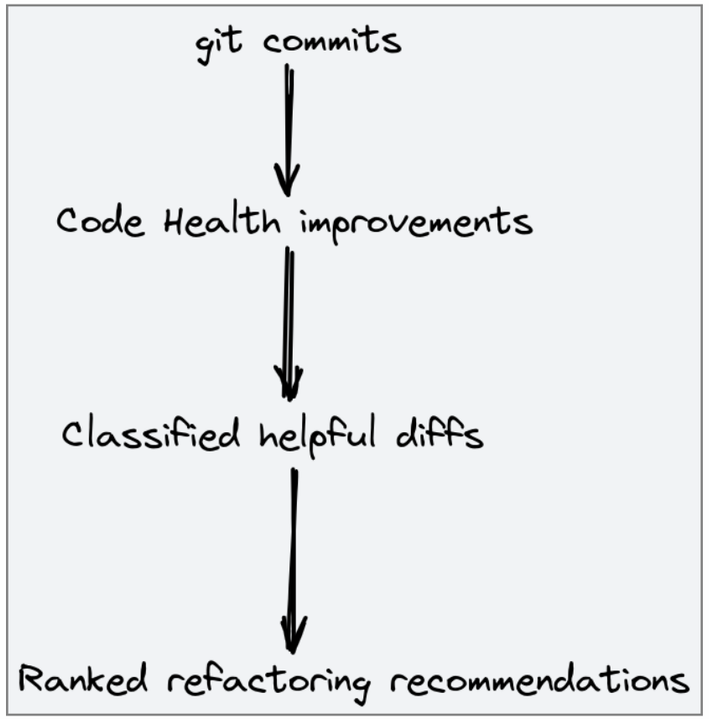
Repeat After Me!

Read about "making repeated edits easier with IntelliCode suggestions", by Peter Groenewegen.

Test Case Generation

Ponicode was alive in 2022, now it's ~~dead~~ acquired by CircleCI.

Automated Refactoring



I found this picture in the CodeScene website.

[Complete PR Review Risks Explain Safurai Repeat Tests Refactor LLM]

Large Language Models (LLM)



Look at this code (it is written in our own object-oriented programming language called EOLANG) and suggest your opinion about the quality of it in terms of readability. Also, if you see any issues, suggest refactoring:

```
[x] > f
  if. > @
    x.lt 2
    x.plus 25
    x.plus 25
```

Based on the code snippet provided, it seems to define a function called `f` that takes an argument `x`, and applies some logic to it based on a conditional statement.

In terms of readability, there are some aspects of the code that could be improved:

1. The use of `>` to indicate the start of the function definition may be confusing to someone who is not familiar with EOLANG. It might be better to use a more descriptive keyword like `function` instead.
2. The use of `if.` and `@` may be confusing to someone not familiar with EOLANG. It's not immediately clear what these symbols mean.
3. The lack of indentation makes it harder to see which statements are part of the conditional block and which are not.
4. The repeated use of `x.plus 25` could be simplified by combining them into a single statement.

Here is a refactored version of the code that addresses these issues:

javascript Copy code

```javascript
function f(x)
  if x.lt(2) {
    x.plus(50)
  }
  x
```

In this version, I have used the `function` keyword to define the function, and `if` to define the conditional statement. I have also used curly braces to indicate the start and end of the conditional block, which makes it easier to see which statements are part of the block. Finally, I have combined the two `x.plus(25)` statements into a single `x.plus(50)` statement.

Chapter #3:

What's Next?

Neural Software Analysis

"Neural Software Analysis"
By Michael Pradel, Satish Chandra
Communications of the ACM, January 2022, Vol. 65 No. 1, Pages 86–96

Watch it.

Electroencephalography (EEG)

"Understanding Programming Expertise: An Empirical Study of Phasic Brain Wave Changes"

By Igor Crk, Timothy Kluthe, Andreas Stefik

ACM Transactions on Computer-Human Interaction, Volume 23, Issue 1, Article No. 2, Pages 1–29

> Programmers are parts of programs

[Neural EEG]